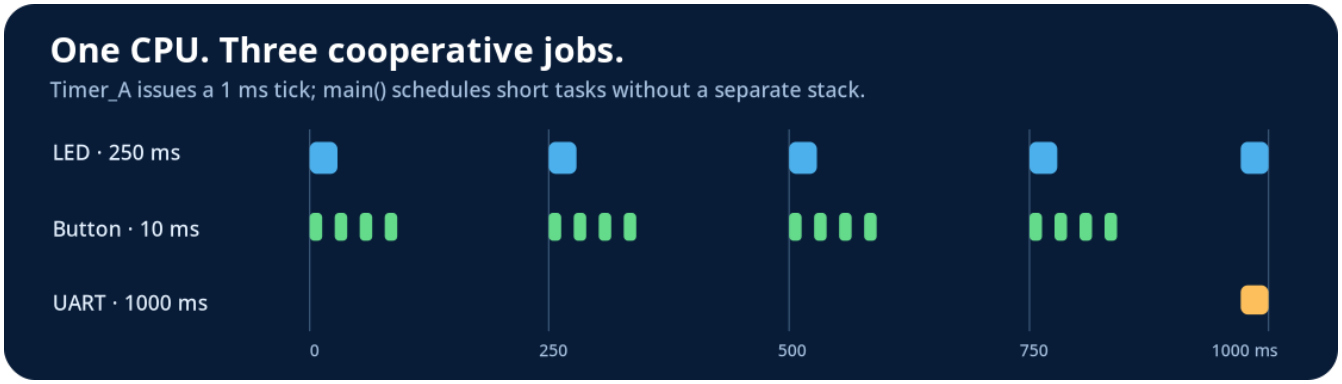
A Tiny Multitasker You Can Understand

From blocking delays to register-level timing, readable scheduling macros, and a measurable low-power experiment.



One CPU executes three short jobs; Timer_A only maintains the millisecond clock.

Overview and hardware

Suggested time: Two class periods, 50–60 minutes each, plus optional EnergyTrace laboratory. Prerequisites: GPIO, switch inputs, Timer_A interrupts, and serial UART. Boards: MSP-EXP430G2 (original) or G2ET (new). EnergyTrace: newer G2ET only.

Stage	Application behavior	Concept revealed
01 · Blocking	LED heartbeat; button scanned only after long delays	Why short presses are lost
02 · Explicit timer	250 ms LED + 10 ms switch + 1000 ms UART	One ISR; three C jobs
03 · RUN_EVERY	Same tasks through a tiny macro/function	Abstraction reveals registers
04 · EnergyTrace	Switch between spin and LPM0 idle	Energy vs. execution style

Ask students

“How can one microcontroller keep an LED blinking, recognize button presses, and send a serial report without a real-time operating system?” Start with student predictions, then compare the actual data. An apparent multitasker is a sequence of short C functions scheduled by elapsed time.

Set up in CCS

Import each stage’s .projectspec file (or create an empty MSP430G2553 C project and copy just that stage’s source files). Stage 1/2 use main.c + edu430.h; Stage 3/4 add tiny_scheduler.h and button_filter.h. Match the LaunchPad programmer and target. For UART, use the LaunchPad Windows COM port at 9600 8N1. The ProjectSpec files need validation in a local CCS installation.

Investigation, Explanation, Transfer

Engage — run blocking Stage 1 (10 minutes)

Ask students to tap the onboard switch quickly during the LED delay. Compare ten intentional short taps to the reported count. What happened? The code checks the button only once each 250 ms busy wait. Button bounce also makes this a weak baseline.

Explore — run explicit Stage 2 (20 minutes)

Show Timer_A's 1 ms interrupt: a 1 MHz SMCLK, TA0CCR0 = 999, and a very short ISR that increments ticks_ms. Main tests 250, 10, and 1000 ms deadlines. Observe better responsiveness. Keep interrupts short; do not place UART printing inside the ISR.

Explain — expand Stage 3 (20 minutes)

Students should open tiny_scheduler.h and write out RUN_EVERY(led_job, now, blink_led). It expands to an if calling job_is_due(), whose unsigned elapsed-time check is (uint16_t)(now - last_run_ms). The blink_led() action still toggles P1OUT with BIT0. A macro makes the idea legible but does not alter the underlying scheduling mechanism.

Extend — deliberate overload (20 minutes)

Change only the LED interval from 250 ms to 100 ms. Observe behavior. Introduce a deliberately slow UART reporting job or temporary long delay; does button response worsen? Remove the delay afterward. Discuss what happens after time counter wrap, skipped deadlines, and long-running sensor transactions.

Evaluate — explain, not memorize (10 minutes)

Have teams submit a drawn timing sequence, an exact macro expansion, and before/after press observations. A good explanation distinguishes "the interrupt increments time" from "main executes jobs."

Data collection worksheet

Trial	Prediction	Observation / explanation
Stage 1: 10 fast taps	_____	_____
Stage 2: 10 fast taps	_____	_____
Stage 3: hold button	_____	_____
Stage 3: slow report task	_____	_____

Measure the Price of Busy-Waiting

On the newer MSP-EXP430G2ET, the built-in eZ-FET EnergyTrace circuit can compare the same tasks under two idle strategies. The G2553 supports EnergyTrace current/energy measurements, not EnergyTrace++ CPU-state tracing.

A controlled comparison

Independent variable: IDLE_LOW_POWER set to 0 (spin) or 1 (LPM0). Controls: same board, clock, calibration, task frequencies, wiring, debugger method, capture duration, and firmware options. Keep POWER_TEST_LED=0 and POWER_TEST_UART=0 in both builds. Dependent variables: average current (µA), energy (µJ or J), and/or average power (µW or W), as provided by CCS.

1. Compile Stage 4 with IDLE_LOW_POWER=0; capture a repeatable baseline.
2. Change only IDLE_LOW_POWER to 1; rebuild and capture the same duration.
3. Repeat both conditions, alternate their order, and note variation.
4. Compute percent current reduction from observed data—not an assumed number.
5. Optional: repeat with POWER_TEST_LED=1 in both builds to see the LED dominate energy.

Why LPM0?

In LPM0, the CPU stops but SMCLK continues. Timer_A uses SMCLK and still fires every 1 ms. The timer ISR must call __bic_SR_register_on_exit(LPM0_bits) so RETI resumes the cooperative loop. LPM3 would stop SMCLK: it is not interchangeable without changing the clock/timer design.

Condition	Mean current (µA)	Energy (µJ)	Observations
Active CPU	_____	_____	_____
LPM0 CPU	_____	_____	_____
Active repeat	_____	_____	_____
LPM0 repeat	_____	_____	_____

Assess (20 points)

4: prediction/data; 4: timer calculation and diagram; 4: accurate RUN_EVERY-to-register expansion; 4: cooperative scheduling and overrun explanation; 4: proposed improvement or controlled EnergyTrace analysis.

Teacher validation reminder

Host GCC syntax and scheduler tests passed, and ProjectSpec XML parses. These checks do not establish that Stage 4 has been compiled in TI CCS or measured on a G2ET. Record actual compiler versions, linker memory usage, debugger revision, measured current, and test date before reporting hardware-verified results.